

CS683 — ADVANCED COMPUTER ARCHITECTURE

Hardware-Conscious Performance Engineering

Programming Assignment 1: 2D Convolution and SGEMM

TODO Name

TODO Roll No.
September 2026

0.1 Experimental setup

Component	Configuration
CPU	13th Gen Intel Core i5-13420H, topology is 4 P-cores + 4 E-cores
L1d cache	48 KiB per P-core
L2 cache	1.25 MiB per P-core
L3 cache	12 MiB
Memory	16 GiB
Vector ISA	SSE, AVX, AVX2, FMA — no AVX-512

1 Task 1: 2D convolution

Baseline: `conv_naive`

1.1 Task 1A: Loop unrolling and reordering

1.2 Task 1B: Tiling

The size of L1d cache in P-core is 48 KiB, and its size in E-Core is 32 KiB, we have used the performant cores through out our assignment.

For the naive variant of Tiling we got an MPKI of 1.099

Metric	Count
Instructions	4,377,401,359
Cycles	920,331,028
L1d cache loads	894,269,791
L1d cache load misses	4,812,940

1.3 Task 1C: SIMD

The number of Instructions in naive 2D convolution is 4,374,266,223

The number of Instructions in SIMD version for 2048 x 2048 matrix with Kernel of dimension 3 x 3 is 4,906,638,833

For this particular configuration of inputs we got a speedup of 7.93

Matrix size	K	Speedup	Instructions	L1d misses	MPKI
256	3	8.24×	79,908,283	133,260	1.668
256	5	9.59×	168,636,071	139,416	0.827
512	3	6.28×	309,934,501	393,169	1.269
512	5	9.02×	664,888,948	466,413	0.701
1024	3	7.52×	1,229,221,723	1,550,748	1.262
1024	5	9.16×	2,648,557,208	1,928,404	0.728

2048	3	7.93×	4,906,638,833	6,841,282	1.394
2048	5	9.10×	10,585,776,892	17,458,830	1.649

Table: For 256 Bit SIMD

Matrix size	K	Speedup	Instructions	L1d misses	MPKI
256	3	5.48×	87,724,922	121,413	1.384
256	5	4.86×	185,825,985	132,613	0.714
512	3	5.62×	340,930,560	415,973	1.220
512	5	5.02×	733,174,587	468,340	0.639
1024	3	4.34×	1,354,016,801	1,640,039	1.211
1024	5	4.72×	2,922,516,815	1,958,168	0.670
2048	3	4.24×	5,403,258,689	7,360,450	1.362
2048	5	4.81×	11,679,133,866	16,389,588	1.403

Table: For 128 Bit SIMD

1.4 Task 1D: Sabka saath sabka vikaas

1.5 Results

Stage	Time (ms)	GFLOP/s	Speedup vs naive
naive			1.00×
reorder			
unroll			
tile			
simd			
optimized			

Table: 2048 × 2048 image, K = 3.

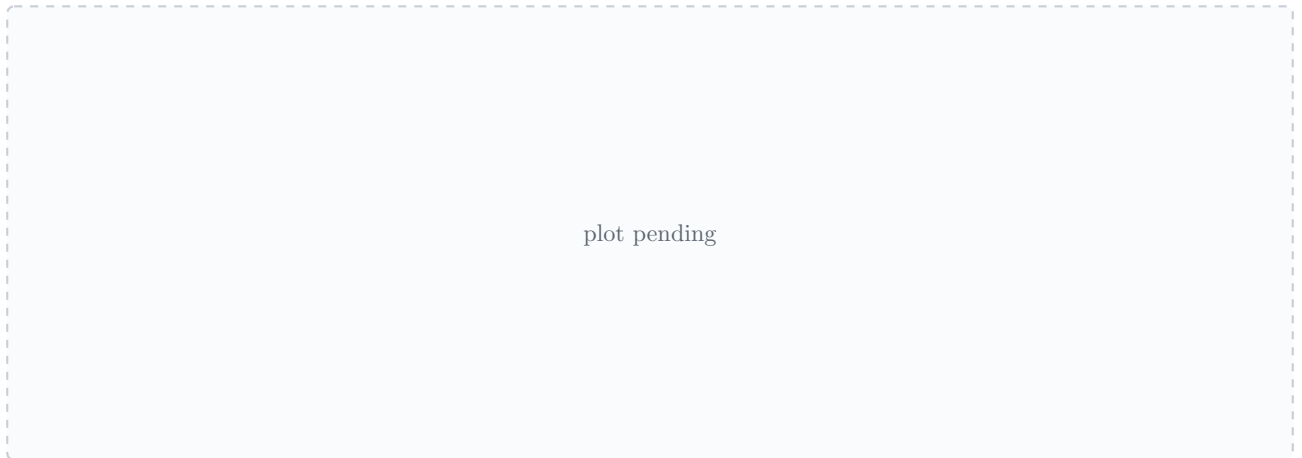


Figure 1: Speedup over `conv_naive` versus image dimension, at fixed $K = 3$.

plot pending

Figure 2: Speedup over `conv_naive` versus kernel size K , at a fixed image size.

1.6 Discussion

2 Task 2: Matrix multiplication (SGEMM)

Baseline: `matmul_naive`

2.1 Task 2A: Software prefetching

2.2 Task 2B: SIMD (Single Instructor Multiple Deadlines)

Matrix Size	SIMD				Without SIMD	
	SIMD width 128		SIMD width 256			
	Instruc-tions (M)	Execution Time	Instruc-tions (M)	Execution Time	Instruc-tions (M)	Execution Time
256	659.68	0.765 ms	645.32	0.478 ms	620.38	8.07 ms
512	5,194.07	7.714 ms	5,057.22	5.724 ms	4,888.86	55.58 ms
1024	41,170.41	76.117 ms	40,104.55	59.397 ms	38,886.06	487.66 ms
2048	328,261.32	1291.929 ms	320,231.07	1103.235 ms	310,339.38	4838.61 ms

Table: Matrix Sizes vs Instruction size & Execution time for SIMD and non SIMD

2.3 Task 2C: Software prefetching + SIMD

2.4 Results

Stage	Time (ms)	GFLOP/s	Speedup vs naive
naive			1.00×
simd			
prefetch			
optimized			

Table: $M = N = K = 1024$.

plot pending

Figure 3: Speedup over `matmul_naive` versus matrix size, $M = N = K$ from 128 to 2048.

plot pending

Figure 4: Speedup of `matmul_prefetch` versus prefetch distance.

plot pending

Figure 5: Speedup of `matmul_prefetch` for the `_MM_HINT_T0` / `T1` / `T2` / `NTA` locality hints.



plot pending

Figure 6: Speedup versus SIMD width: 128-bit SSE, 256-bit AVX2, 512-bit AVX-512.

2.5 Integration with llama.cpp

2.6 Discussion

3 Conclusion